

Access Control Implementation

CompTIA SecAI+: AI Security Certification Complete Exam Prep 2026 | Section 16: Securing AI Systems in Production

1. Why AI Systems Require Specialized Access Controls

Standard enterprise access controls were designed for applications that retrieve and display data. AI systems are fundamentally different: they do not merely retrieve information — they synthesize, reason over, and amplify it. An analyst who queries a security AI model may receive an output that combines threat intelligence, behavioral analysis, and pattern recognition in ways that would require hours of manual research without AI assistance. This amplification effect means that unauthorized AI access does not carry the same risk profile as unauthorized database access.

Additionally, AI systems often serve as intermediaries that inform consequential security decisions: which alerts to investigate, which systems to patch first, which user accounts to restrict. A compromised AI access path can therefore influence not just what an attacker learns, but what the security team does in response. This threat profile makes AI access control a security-critical domain requiring deliberate, layered implementation.

Core Principle: AI access controls must address three dimensions: who can access which models, what they can do within those models, and from where and when they can connect. All three dimensions must be enforced simultaneously.

2. Authentication: Establishing Identity Before Granting Access

Authentication answers the question 'who are you?' before any access decision is made. For AI systems that process sensitive security data, passwords alone are insufficient — they can be phished, brute-forced, or stolen. Multi-factor authentication (MFA) combines something the user knows (password) with something they have (TOTP code, hardware token, push notification) to provide defense against credential theft.

MFA requirements for AI systems should be non-negotiable for sensitive deployments. The implementation approach varies by use case:

- Human users — TOTP authenticator apps (Google Authenticator, Microsoft Authenticator) or hardware security keys (FIDO2/WebAuthn) are the standard options. FIDO2 keys are phishing-resistant and preferred for high-privilege access.
- Service accounts and automated systems — password-based authentication is inappropriate; use certificate-based authentication or short-lived API tokens issued by an identity provider. Never use static, long-lived credentials for automated access.
- Third-party integrations — OAuth 2.0 with short token expiry and narrow scope grants; review and rotate regularly.

Authentication events — successful logins, failed attempts, MFA challenges, and password resets — should be logged and sent to the SIEM for anomaly detection. A burst of failed login attempts against an AI system is an early indicator of credential stuffing or brute-force attacks.

3. Role-Based Access Control (RBAC): Structuring Permissions by Job Function

RBAC simplifies access management by grouping permissions into roles aligned with organizational job functions. Individual users are assigned to roles, and roles carry the permissions. When an employee changes departments or leaves the organization, a single role change or role removal updates all their AI system permissions simultaneously.

Role	Permitted AI Access	Typical User
SOC Analyst (Junior)	Query threat detection models; view alert summaries	Entry-level analyst, limited tenure
SOC Analyst (Senior)	All junior permissions + access to threat correlation AI, behavioral analytics	Experienced analyst with established trust
Threat Intelligence Analyst	Threat intel AI models, attribution models, dark web monitoring AI	Specialized intelligence team member
SOC Manager	All analyst permissions + dashboard AI, reporting AI, team metrics	Team lead or shift supervisor
AI System Administrator	Model configuration, user and role management, audit log access	Platform operations team
Security Architect	Development and test model access, architecture design tools	Senior engineer, design authority

Role design principle: roles should reflect actual job functions, not seniority alone. A senior employee who has moved from the SOC to a non-security role should not retain SOC analyst permissions. Roles must be reviewed and updated when job functions change, not only when employees leave.

4. Attribute-Based Access Control (ABAC): Fine-Grained Contextual Decisions

RBAC works well for relatively stable job functions but struggles with context-dependent access decisions. ABAC extends beyond roles by evaluating multiple attributes simultaneously before granting access.

Example ABAC policy for a classified threat attribution model: access is granted only when ALL of the following conditions are met simultaneously:

- User attribute: department = 'Threat Intelligence' AND clearance_level >= 'SECRET'
- Resource attribute: model_classification = 'SENSITIVE' AND data_type = 'threat_attribution'
- Environment attribute: network_location = 'corporate_network' OR vpn_connected = TRUE
- Temporal attribute: access_time BETWEEN '07:00' AND '20:00' local_timezone
- Device attribute: device_compliance_status = 'compliant' AND device_os_patch_level >= 'current-minus-1'

ABAC is more powerful than RBAC but more complex to implement and maintain. The policy engine must evaluate multiple attributes in real time for every access request. For AI systems processing highly sensitive data — classified intelligence, regulated personal information, critical infrastructure data — the additional precision of ABAC justifies the implementation complexity.

5. Least Privilege and the Problem of Permission Creep

The principle of least privilege states that users, services, and automated processes should have exactly the permissions required to perform their current responsibilities — no more. In practice, the challenge is not initial implementation but long-term maintenance.

Permission creep occurs when users accumulate permissions over time through role changes, project assignments, and organizational restructuring without corresponding permission removals. An analyst who temporarily joined a high-sensitivity project six months ago may still hold the elevated permissions granted for that project, even though their current role does not require them.

Real-World Incident — SolarWinds Supply Chain Attack (2020): Post-incident analysis revealed that the attackers were able to move laterally through SolarWinds' environment partly because service accounts and users had accumulated excessive privileges over time. Accounts held permissions far beyond what their current functions required, enabling attackers to access sensitive systems and data without triggering expected anomalies. The incident is a canonical example of how permission creep in non-AI systems enables lateral movement — the same dynamic applies with amplified impact in AI systems that have access to synthesized intelligence across multiple data sources.

Combating permission creep requires systematic processes: quarterly access reviews for all AI system permissions, automated alerts when users have held elevated permissions for more than a defined period without access activity, and formal offboarding checklists that explicitly remove AI system access.

6. Model-Level and Feature-Level Access Controls

Access control in AI environments operates at two granularities that are often conflated but serve distinct purposes.

Model-level access controls determine which AI models a user may interact with. A SOC analyst may be authorized to use the malware classification model but not the threat attribution intelligence model. Model-level controls are typically implemented through RBAC or ABAC policies enforced at the API gateway before requests are forwarded to the inference service.

Feature-level permissions go deeper: they control what a user can do within a model they are authorized to access. Even an authorized user should not have unconstrained capabilities. Feature-level permission categories in security AI deployments:

Feature	Description	Typical Authorized Roles
Query / Inference	Submit prompts and receive model responses	All authorized users
View results	Access model outputs and	All authorized users

	recommendations	
Export outputs	Download or forward model outputs to external systems	Senior analysts, managers
Configure parameters	Adjust model thresholds, sensitivity settings	AI administrators, architects
Retrain / fine-tune	Submit training data or trigger model updates	AI administrators, data science team
Audit log access	View access history and usage logs	AI administrators, compliance team

7. API Key Management for Automated and Service Access

AI systems that expose APIs are accessed not only by human users but by automated workflows, monitoring platforms, and integration pipelines. These non-human access paths require API key management as a distinct security discipline.

API key management best practices:

- Unique keys per application — each application or service that calls the AI API receives its own API key with permissions scoped to exactly what that application needs. Never share a single key across multiple applications.
- Narrow scope — apply the principle of least privilege to API key permissions. A threat monitoring integration needs inference access, not model management or user administration.
- Expiration and rotation — API keys should have defined expiration dates. A common standard is 90-day rotation cycles for sensitive integrations, with automated alerts when keys approach expiration.
- Revocation capability — the organization must be able to instantly revoke a compromised API key without disrupting other integrations. This requires the unique-key-per-application architecture.
- Usage monitoring — track API call volume, error rates, and access patterns per key. Anomalies in API usage — a sudden volume spike, access at unusual times, errors from unexpected endpoints — may indicate key compromise.

8. Network-Based and Time-Based Access Controls

Network-based access controls restrict which network locations can reach AI system endpoints. Even valid credentials from the correct user cannot establish a connection if the request originates from an unauthorized network segment.

Network control implementation layers for AI systems:

- Corporate network / VPN requirement — internal security AI models should be accessible only from the corporate network or through authenticated VPN connections. This ensures remote access occurs over a secured channel.
- Network segmentation — highly sensitive AI models (classified intelligence, critical infrastructure analysis) may be isolated in dedicated network segments accessible only from specific approved workstations or jump servers.

- DMZ placement for external services — AI models exposed to external partners or customers should be deployed in a DMZ with strict inbound and outbound filtering.
- IP allowlisting for service integrations — automated systems that call AI APIs should connect from known, static IP ranges that are explicitly allowlisted at the network layer.

Time-based access controls complement network controls by restricting when AI systems are accessible. Standard users may be authorized only during business hours; administrators may have extended windows. Unusual access attempts outside authorized time windows should trigger alerts regardless of credential validity — a valid login at 3:00 AM from an account that has never accessed the system outside business hours is an anomaly worth investigating.

9. Access Request Workflows and Session Management

Formal access request workflows add accountability and auditability to the permission-granting process. Rather than granting access informally, users submit structured requests that are reviewed and approved by designated approvers — typically the user's manager combined with the AI system owner or security team.

A well-designed access request for AI systems captures: the specific model or feature requested, the business justification, the intended duration of access, and the approver's signature. Time-limited access grants are particularly effective for temporary project needs — the permission expires automatically, eliminating the need for manual revocation and preventing accumulation.

Session management controls secure active sessions after authentication has succeeded:

- Inactivity timeout — sessions automatically terminate after a defined idle period (typically 15–30 minutes). Prevents unattended workstation access.
- Maximum session duration — even active sessions expire after a defined maximum (typically 8 hours for standard users, shorter for highly sensitive models). Requires reauthentication rather than allowing indefinitely persistent sessions.
- Single active session — some environments limit each user to one concurrent session. This prevents credential sharing and makes it easier to detect session hijacking.
- Reauthentication for sensitive actions — exporting model outputs, changing configuration, or accessing particularly sensitive models may require step-up authentication even within an active session.

10. Continuous Monitoring and Periodic Access Reviews

Monitoring transforms access control from a static gate into an active defense mechanism. Key events to monitor and alert on in AI access control systems:

Event Type	Alert Trigger	Suggested Response
Authentication anomaly	Failed logins > 5 in 10 min; login from new country	Lock account; require MFA re-enrollment; investigate
Access time anomaly	Access outside authorized hours for account type	Alert SOC; verify with user; investigate if unconfirmed
Feature scope escalation	User accesses feature beyond their	Block and alert; review for

	role permissions	permission misconfiguration
API key anomaly	Volume spike > 3x baseline; access from unlisted IP	Suspend key; investigate for compromise
Privilege escalation attempt	Request for elevated permissions without workflow	Block; alert security team; review user activity
Inactive account access	Account not used in > 30 days suddenly becomes active	Verify with account owner; investigate for compromise

Periodic access reviews — conducted quarterly for most security AI environments — systematically verify that every permission grant remains justified. The review process examines each user's current role and compares it against their AI system permissions, flagging discrepancies for remediation. Former employees, role-changers, and project completers are the primary sources of permissions that should be removed.

Key Terms Glossary

Term	Definition
Multi-Factor Authentication (MFA)	An authentication method combining two or more verification factors (something known, something possessed, something inherent) to confirm user identity.
Role-Based Access Control (RBAC)	An access control model where permissions are grouped into roles aligned with job functions and users are assigned to roles rather than individual permissions.
Attribute-Based Access Control (ABAC)	An access control model that evaluates multiple attributes (user, resource, environment, time) simultaneously to make fine-grained access decisions.
Least Privilege	The security principle that users and systems should hold only the minimum permissions necessary to perform their current responsibilities.
Permission Creep	The gradual accumulation of excessive permissions as users change roles, join projects, and receive access grants without corresponding revocations.
Model-Level Access Control	Access control that determines which AI models a user is authorized to interact with, enforced at the API gateway or authentication layer.
Feature-Level Permission	Access control within an authorized model that restricts what capabilities a user may exercise (query, export, configure, retrain).
API Key	A unique credential issued to an application or service for programmatic access to an AI API, scoped to minimum required permissions with defined expiration.
Access Request Workflow	A formal process requiring users to submit documented, approved requests for AI system access, creating audit trails and enforcing accountability.
Session Management	Controls governing the lifetime and security of active user sessions, including inactivity timeouts, maximum duration, and

	reauthentication requirements.
Access Review	A periodic process in which administrators verify that every current AI system permission grant remains appropriate and justified by the user's current role.

Quick Revision Checklist

- AI access controls must address three dimensions: who, what (actions), and where/when — all three enforced simultaneously.
- MFA is mandatory for human access to sensitive AI systems; service accounts use certificate or token-based auth, never static passwords.
- RBAC groups permissions by job function and simplifies management; ABAC evaluates multiple attributes for fine-grained context-dependent decisions.
- Least privilege requires periodic enforcement — permission creep is a natural organizational tendency that must be actively countered.
- Model-level controls determine which models a user may access; feature-level controls determine what they may do within an authorized model.
- API keys must be unique per application, narrowly scoped, time-limited with rotation cycles, and monitored for usage anomalies.
- Network controls restrict access by network location; time-based controls restrict access by time of day — both complement identity-based controls.
- Access request workflows add accountability and create audit trails; time-limited grants prevent accumulation of temporary permissions.
- Session management includes inactivity timeouts, maximum session duration, and step-up authentication for sensitive operations.
- Continuous monitoring detects anomalies in real time; quarterly access reviews catch stale permissions that monitoring misses.

10 Exam-Based MCQs — Access Control Implementation

Test yourself after reading. These questions are written in SecAI+ exam style — scenario-heavy with plausible distractors. Read every explanation including for questions you answered correctly.

Q1. *Scenario:* A defense contractor is deploying an AI system to analyze classified threat intelligence for government clients. The initial design specifies: username and password login, RBAC with four defined roles, and API keys for automated data ingestion pipelines. The security architect is reviewing the design for gaps. A security architect is designing access controls for a new AI-powered threat intelligence platform that will process classified government data. Standard employee login credentials plus role-based permissions have been proposed. What CRITICAL control is missing from this design?

- A) Rate-based guardrails on the AI model's output
- B) Multi-factor authentication for all users accessing the platform
- C) A dark theme interface to reduce analyst eye strain during extended use
- D) API key rotation on a 180-day cycle

Answer: B **Explanation:** B is correct because a platform processing classified data requires strong identity assurance, and passwords alone are insufficient against credential theft, phishing, and brute-force attacks. MFA is a foundational requirement for sensitive AI deployments, not an optional enhancement. A is wrong because rate-based guardrails are a model behavior control, not an access control — they address what the model produces, not who can reach it. They are important but not the critical missing element in an access design discussion. C is wrong because interface design is not a security control — it is irrelevant to the access control design question. D is wrong because 180-day API key rotation is less frequent than the recommended 90-day standard for sensitive systems, and in any case the question identifies what is missing, not what could be improved — MFA is the absent critical control.

Q2. Scenario: During a quarterly access review, the AI system administrator discovers that a mid-level SOC analyst holds query and export permissions on the sensitive threat attribution AI model. Investigation reveals these permissions were granted six months ago for a special project that concluded four months ago. The analyst is currently assigned to standard tier-2 alert investigation duties. A SOC analyst who previously worked on a classified threat attribution project for six months has since returned to standard SOC duties. An access review reveals they still hold access to the threat attribution AI model from their project assignment. What BEST describes the security risk and correct remediation?

- A) No risk exists because the analyst is a trusted employee; no action is required.
- B) This is permission creep; the analyst's access to the threat attribution model should be revoked to restore least privilege.
- C) The analyst should retain access in case they are assigned to a similar project in the future.
- D) Access should be downgraded to read-only to reduce risk while preserving project continuity.

Answer: B **Explanation:** B is correct because this is a textbook example of permission creep — the analyst accumulated elevated access during a temporary project and retained it after returning to a role that does not require it. The least privilege principle requires removing access that is no longer justified by current responsibilities. A is wrong because trust level is not a substitute for least privilege enforcement — insider threat risk, account compromise risk, and accidental misuse risk all apply to trusted employees. Retaining unnecessary access creates an attack surface regardless of the employee's intentions. C is wrong because speculative future need is not a valid justification for current access — if the analyst is assigned to a future project, access should be granted at that time through a formal access request. D is wrong because the analyst's current role does not require any access to the threat attribution model; read-only access to a model they do not need is still a least-privilege violation, just a smaller one.

Q3. An organization wants to restrict access to a classified threat attribution AI model so that it is only accessible to users who are in the Threat Intelligence department, hold at least a SECRET clearance, are connecting from the corporate network, and are accessing during approved business hours. Which access control model is BEST suited to enforce all of these conditions simultaneously?

- A) Role-Based Access Control (RBAC)
- B) Mandatory Access Control (MAC) based on data classification labels only
- C) Attribute-Based Access Control (ABAC)
- D) Discretionary Access Control (DAC) delegated to the model owner

Answer: C **Explanation:** C is correct because ABAC is specifically designed to evaluate multiple attributes simultaneously — user department, clearance level, network location, and time of day — before granting access. The policy described requires all four conditions to be true at the same time, which is the defining use case for ABAC. A is wrong because RBAC groups permissions by role (department or job function) but does not natively evaluate real-time contextual attributes like network location and access time within the same policy decision — you would need additional controls layered on top of RBAC. B is wrong because MAC based solely on classification labels would enforce clearance requirements but would not address the network location and time constraints described. D is wrong because DAC delegates access decisions to resource owners and does not provide the systematic, policy-driven evaluation of multiple attributes required here.

Q4. Which of the following BEST represents the least privilege principle applied to AI system API keys?

- A) One master API key shared by all applications connecting to the AI system, rotated annually.
- B) A unique API key per application, scoped to only the permissions that application requires, with 90-day rotation.
- C) Static API keys with no expiration, stored in application environment variables.
- D) API keys issued to individual developers who share them across all applications they manage.

Answer: B **Explanation:** B is correct because least privilege for API keys requires unique keys per application (isolating the blast radius if one is compromised), narrow permission scope (the application only gets the permissions it genuinely needs), and regular rotation (limiting the window during which a compromised key can be used). A is wrong because sharing a single master key across all applications violates both least privilege (too broad a scope) and isolation (compromising one application compromises all). Annual rotation is also insufficiently frequent for sensitive AI system access. C is wrong because static keys with no expiration are indefinitely valid if compromised — rotation is a fundamental key hygiene requirement. Environment variables alone are not secure storage for credentials. D is wrong because keys should be assigned to applications (services), not individuals — developer-owned shared keys create accountability gaps and propagate when developers change projects or leave the organization.

Q5. A security AI system logs show that a user account that normally accesses the system between 8 AM and 6 PM made 47 consecutive authentication attempts between 2:30 AM and 3:15 AM, all from a new geographic location, before ultimately authenticating successfully at 3:17 AM. Which access control response is MOST appropriate?

- A) Log the event and review it during the next scheduled weekly security log review.
- B) Immediately suspend the account pending investigation; alert the SOC team.
- C) Block future access from the new geographic location only.
- D) Require the user to change their password at next login.

Answer: B **Explanation:** B is correct because the combination of anomalies — access at a highly unusual hour, 47 failed attempts (characteristic of credential stuffing or brute force), a new geographic location, and ultimately a successful authentication after the failed attempts — strongly suggests account compromise. The appropriate response to a suspected compromise is immediate account suspension to prevent further unauthorized access, combined with SOC notification to begin investigation and the user

verification process. A is wrong because deferring review of a potential compromise to a weekly log review cycle allows the attacker to operate in the compromised account for days — immediate response is required for this level of anomaly. C is wrong because blocking only the geographic location does not address the possibility that the attacker has already authenticated and established persistence, and does not prevent access from other attacker-controlled locations. D is wrong because a password change alone is insufficient if the account is already compromised — the active session (authenticated at 3:17 AM) may still be in use, and the investigation must precede or accompany any remediation.

Q6. Which session management control MOST directly mitigates the risk of an analyst who forgets to log out of an AI security platform on a shared workstation in the SOC?

- A) Maximum session duration of 8 hours
- B) Inactivity timeout that terminates sessions after 15–30 minutes of idle time
- C) Single active session per user account
- D) MFA requirement at initial login

Answer: B **Explanation:** B is correct because an inactivity timeout directly addresses the scenario of a forgotten or unattended session — if the analyst walks away without logging out, the session terminates automatically after the idle period, preventing the next person to use the workstation from accessing the AI platform under the previous analyst's identity. A is wrong because a maximum session duration of 8 hours would not protect the workstation during the hours between the analyst leaving and the session expiring — the exposure window is the idle period, not the total session duration. C is wrong because a single active session per user prevents concurrent sessions but does not help when the abandoned session is the only one — it does not terminate idle sessions. D is wrong because MFA at login strengthens initial authentication but has no effect after the session is established; once authenticated, the unattended session remains open regardless of MFA.

Q7. An organization is implementing feature-level permissions for its AI-powered SIEM correlation platform. A tier-2 SOC analyst needs to query the AI for alert correlation insights and view recommendations, but should not be able to modify correlation rules or export raw intelligence data. Which permission set CORRECTLY applies least privilege for this role?

- A) Query, View, Export, Configure
- B) Query, View only
- C) Configure and Retrain only
- D) All permissions with read-only flag on configuration

Answer: B **Explanation:** B is correct because the analyst's stated needs are to query the AI and view its recommendations — these are the only features required. Exporting raw intelligence data and configuring correlation rules are explicitly outside their responsibilities, so these permissions should not be granted. A is wrong because including Export and Configure permissions violates least privilege — these capabilities are not required for the analyst's duties and increase the potential impact of a compromised account. C is wrong because Configure and Retrain permissions are administrative functions; this set would not allow the analyst to perform their primary function (query and view) while granting capabilities they do not need. D is wrong because a read-only flag on configuration is not equivalent to no access — read-only

configuration access may still expose sensitive security rule details, and the permission model should exclude configuration entirely, not just limit it.

Q8. Why are time-limited access grants particularly effective for managing temporary project-based AI system access?

- A) They automatically expire when the project ends, eliminating the need for manual revocation and preventing permission accumulation.
- B) They allow users to access any AI model during the project period regardless of their role.
- C) They bypass the normal access request approval workflow for faster project onboarding.
- D) They prevent the AI model from being retrained during the project period.

Answer: A **Explanation:** A is correct because time-limited grants address the root cause of permission creep: permissions that should be temporary become permanent due to operational oversight or lack of a revocation process. When a grant expires automatically at a defined date, the permission is removed without requiring anyone to remember to do it — the system handles cleanup. This is especially effective for project-based access where the end date is known in advance. B is wrong because time-limited grants do not expand permissions beyond what the access request specifies — they still grant only the permissions approved for the specific project, scoped to the user's needs. C is wrong because time-limited grants are processed through the normal access request workflow — the only difference is that the grant includes an expiration date. They do not bypass approval processes. D is wrong because grant duration controls user access to the model, not the model's training or operational lifecycle — these are completely separate concerns.

Q9. An organization conducts quarterly access reviews for its AI security platform. What is the PRIMARY security goal of these reviews?

- A) To identify and remove permissions that are no longer justified by users' current roles, preventing permission creep.
- B) To test the AI model's performance and accuracy against current threat data.
- C) To audit the AI model's training data for bias and fairness issues.
- D) To review and update the AI system's detection rule sets for new threat techniques.

Answer: A **Explanation:** A is correct because periodic access reviews specifically target the staleness problem in access control: permissions that were appropriate when granted but no longer match the user's current role due to promotions, transfers, project completions, or organizational changes. Quarterly reviews provide a systematic mechanism to identify and remediate these stale permissions before they can be exploited. B is wrong because model performance evaluation is a separate AI governance activity — it assesses whether the model's recommendations are accurate, not whether human access permissions are appropriate. C is wrong because bias and fairness auditing is an AI ethics and governance activity unrelated to access control review processes. D is wrong because detection rule updates are a security content management activity — they update what the model looks for, not who is authorized to access the system.

Q10. A security team is selecting between RBAC and ABAC for controlling access to a new AI threat intelligence system. The system serves users across multiple clearance levels, physical locations, and device types, and access decisions need to vary based on the sensitivity of the specific intelligence dataset being queried. Which recommendation is MOST appropriate?

- A) Implement RBAC only — it is simpler to manage and sufficient for all use cases.
- B) Implement ABAC — it can evaluate user clearance, device compliance, location, and data sensitivity simultaneously to make context-appropriate decisions.
- C) Implement DAC — allow model owners to individually approve each access request.
- D) Implement RBAC with a manual override process for context-specific exceptions.

Answer: B **Explanation:** B is correct because the described requirements — multiple clearance levels, location-dependent access, device-type considerations, and per-dataset sensitivity evaluation — are exactly the contextual, multi-attribute decision requirements that ABAC is designed for. No single role can capture all of these combinations; the access decision must evaluate all attributes simultaneously. A is wrong because RBAC cannot natively evaluate real-time contextual attributes like location, device compliance status, or dataset-specific sensitivity. Layering enough roles to cover all combinations would result in role explosion — an unmanageable proliferation of narrow roles that defeats RBAC's administrative simplicity advantage. C is wrong because DAC delegates access decisions to individual resource owners, which creates inconsistency, does not scale, and provides no systematic enforcement of clearance or location requirements. D is wrong because RBAC with manual overrides is operationally unsustainable at scale — the volume of context-specific exceptions would require constant manual intervention and provides a weaker security posture than a properly implemented ABAC policy engine.